



OpenSpec: Spec-Driven Development nell'era degli Agenti AI

Un nuovo paradigma per la collaborazione tra Umani e Intelligenza Artificiale

Matteo Baccan

DevFest Modena 2026 - 3/4 Ottobre 2026, Modena

Track: AI & Machine Intelligence



Google
Developer
Groups

Di cosa parleremo

01 **Dal Vibe Coding a OpenSpec**

Perché i prompt improvvisati non scalano e serve una struttura

03 **Il Linguaggio dell'Intento**

EARS + BDD: specifiche che l'agente non può fraintendere

05 **Dal Team al Legacy**

Adozione incrementale sul brownfield e anti-pattern da evitare

02 **La Meccanica di OpenSpec**

Directory, artefatti e Delta Specs: l'anatomia del framework

04 **Il Ciclo di Esecuzione**

Propose → Apply → Archive: la macchina a stati delle modifiche

06 **Perché Tutto Questo Conta**

Determinismo, intento persistente e il ruolo dell'Agentic Engineer



Google
Developer
Groups

01

Dal Vibe Coding a OpenSpec

Perché i prompt improvvisati non scalano e serve una struttura



Google
Developer
Groups

Il Problema: Agenti AI senza struttura

Gli agenti AI sono collaboratori attivi, ma senza struttura generano **vibe coding**: prompt informali, contesto volatile, drift dei requisiti.

- I requisiti vivono solo nella chat: nessuna documentazione persistente
- Allucinazioni architetturali e scelte inconsistenti
- Debito tecnico invisibile e difficilmente tracciabile



Google
Developer
Groups

La Soluzione: Spec-Driven Development

Lo **Spec-Driven Development** inverte il paradigma: **la struttura prima del codice**.

	Vibe Coding	Spec-Driven Development
Flusso	Prompt → Codice	Idea → Specifica → Architettura → Task → Codice
Artefatti	Nessuno (solo chat)	<code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs
Verificabilità	Impossibile	Ogni fase produce output controllabile



Google
Developer
Groups

Cos'è OpenSpec?

OpenSpec è un framework open-source progettato per la Spec-Driven Development.

Non richiede paywall o API key proprietarie, non è vincolato a un IDE specifico o a un singolo vendor AI.

È un sistema basato su file Markdown che funge da "controllo di versione per l'intento"

- **Zero lock-in**
- Specifiche portabili
- Valore che resta nel repository.



Google
Developer
Groups

OpenSpec: i tre pilastri

Pilastro 1: Brownfield-First ($1 \rightarrow n$)

Ottimizzato per l'evoluzione di codebase esistenti, non solo per prototipi greenfield ($0 \rightarrow 1$).

Permette di modificare logiche complesse in sicurezza, senza riscrivere tutto da zero.

Pilastro 2: Architettura Leggera

Markdown + Git come base operativa: niente infrastruttura pesante, niente database complessi.

Il metodo resta trasparente, versionabile e vicino al flusso reale del team.

Pilastro 3: Agnosticismo Totale

Zero lock-in verso IDE, modelli o vendor: il valore vive nelle specifiche, non nella piattaforma.

Se cambi agente o ambiente, il processo resta intatto perché l'intento rimane nel repository.



Google
Developer
Groups

La specifica è la fonte della verità

I documenti sono **istruzioni eseguibili e vincolanti** per gli agenti AI, non suggerimenti: solo intento deterministico, espresso in un contratto scritto.

- **Living documentation:** le specifiche vivono in Git accanto al codice e si evolvono col sistema, senza diventare obsolete
- A differenza dei ticket (Jira, Linear), stanno dove l'agente opera: **nel repository**
- Supporta oltre 30 strumenti: cambi modello, IDE o vendor e il processo resta intatto

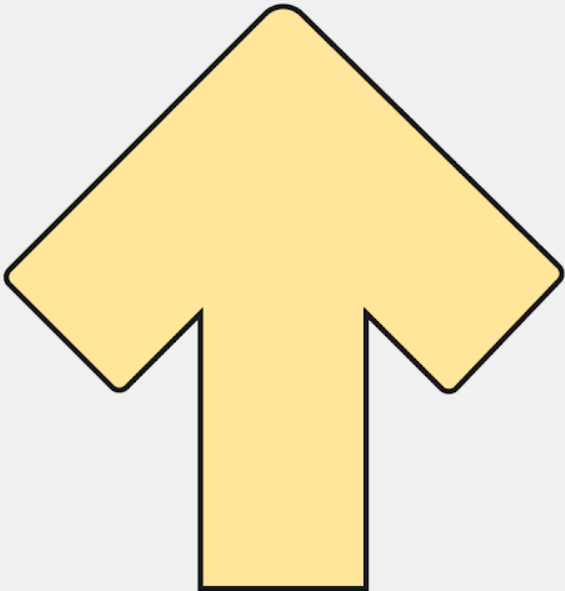


Google
Developer
Groups

02

La Meccanica di OpenSpec

Directory, artefatti e Delta Specs:
l'anatomia del framework



Google
Developer
Groups

Un'unica directory: la memoria dell'agente

Tutto risiede in `openspec/`, la memoria a lungo termine dell'agente AI, con due anime separate:

Cartella	Ruolo	Contenuto
<code>specs/</code> (l'Essere)	La Source of Truth	Comportamento attuale del sistema, per domini logici (<code>auth/</code> , <code>payments/</code> , <code>ui/</code>)
<code>changes/</code> (il Divenire)	Il workspace di modifica	Una cartella isolata per ogni feature o bug fix, senza conflitti finché non si archivia

Questa distinzione garantisce transizioni sicure e modifiche parallele.



Google
Developer
Groups

Governance attiva: come `config.yaml` guida ogni richiesta

<code>config.yaml</code>	Funzione	Effetto sull'agente
<code>schema</code>	Definisce workflow e struttura degli artefatti attesi	L'agente non improvvisa formati: produce output conformi
<code>context</code>	Fornisce architettura globale, stack, standard tecnici	Ogni piano parte dagli stessi vincoli di progetto
<code>rules</code>	Impone guardrail granulari per fase e artefatto	Riduce deviazioni, allucinazioni e scelte fuori policy

Risultato operativo: il file non resta passivo nel repository, viene iniettato nel prompt di pianificazione e rende la conformità tecnica sistematica.



Google
Developer
Groups

Anatomia end-to-end di una proposta

Fase	File	Domanda chiave	Ruolo operativo
1	proposal.md	Perché / Cosa stiamo cambiando?	Definisce intento strategico e ambito lavori (business case iniziale).
2	design.md	Come lo realizziamo?	Fissa decisioni tecniche: architettura, flussi dati, librerie e vincoli.
3	tasks.md	Come eseguiamo in modo verificabile?	Scompone in task atomici e sequenziali, eseguibili dall'agente senza ambiguità.
4	Delta Specs	Cosa cambia nella Source of Truth?	Applica patch ai requisiti con sezioni ADDED , MODIFIED , REMOVED .

Flusso completo: proposal.md → design.md → tasks.md → Delta → consolidamento → specs/



Google
Developer
Groups

L'intento in pratica: `proposal.md`

Cattura l'intento strategico: "perché stiamo facendo questa modifica?". È il punto di allineamento iniziale tra umano e macchina.

Why

Gli utenti abbandonano la registrazione: serve il login social.

What Changes

- Aggiunta autenticazione OAuth (Google) al flusso di login
- Nessun impatto sul login email/password esistente



Google
Developer
Groups

L'esecuzione in pratica: `tasks.md`

Una checklist di azioni atomiche: l'agente aggiorna le spunte in tempo reale mentre scrive il codice, garantendo totale trasparenza.

1. Backend

- [x] 1.1 Aggiungere endpoint GET /auth/google
- [x] 1.2 Gestire il callback e creare l'utente
- [] 1.3 Test di integrazione del flusso completo



Google
Developer
Groups

Delta Specs: patch per i requisiti

La vera innovazione: invece di riscrivere l'intera specifica, l'agente descrive solo cosa cambia: meno token, più accuratezza.

- **ADDED Requirements**: comportamenti completamente nuovi
- **MODIFIED Requirements**: requisiti riscritti per intero, senza perdere regole pregresse
- **REMOVED Requirements**: funzionalità deprecate, con motivo e migrazione

MODIFIED Requirements

Requirement: Session Duration

The system SHALL scadere la sessione dopo 30 minuti di inattività (prima: 24 ore).



Google
Developer
Groups

03

Il Linguaggio dell'Intento

EARS + BDD: specifiche che l'agente non può fraintendere



Google
Developer
Groups

Il linguaggio dell'intento: EARS + BDD

Asse	EARS (obbligazione)	BDD (esecuzione)
Scopo	Ridurre ambiguità nei requisiti	Rendere verificabili i comportamenti
Forma	Parole chiave normative: SHALL , MUST , SHOULD	Struttura scenario: GIVEN , WHEN , THEN
Domanda a cui risponde	"Cosa è obbligatorio?"	"Come si osserva il risultato?"
Esito pratico	Contratti chiari per l'agente	Test di accettazione derivabili

Regola operativa: prima definiamo il vincolo con EARS, poi ne proviamo l'esecuzione con BDD.



Google
Developer
Groups

Dallo scenario al test

Le specifiche descrivono il comportamento osservabile, non i passi di implementazione: sono contratti di business, non tutorial di codice.

- **GIVEN:** Il contesto iniziale (es. "Dato un utente non autenticato").
- **WHEN:** L'azione scatenante (es. "Quando visita la dashboard").
- **THEN:** Il risultato atteso (es. "Allora viene reindirizzato al login").

Con scenari così strutturati, l'agente genera automaticamente i test unitari o e2e corrispondenti, chiudendo il ciclo della qualità.

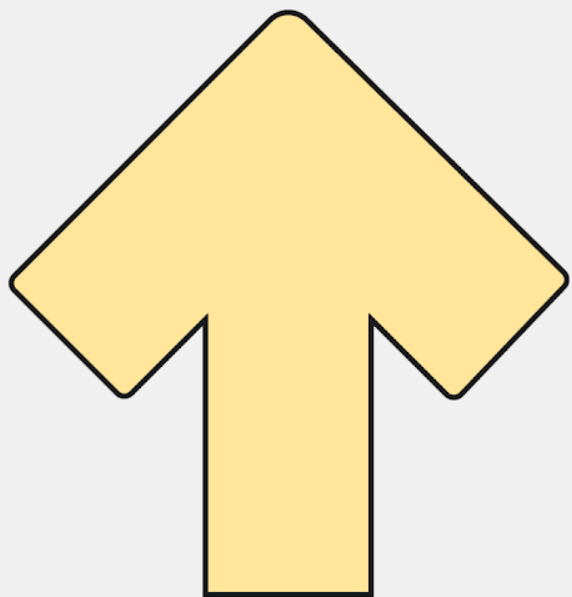


Google
Developer
Groups

04

Il Ciclo di Esecuzione

Propose → Apply → Archive: la macchina a stati delle modifiche



Google
Developer
Groups

La macchina a stati di OpenSpec

Fase	Obiettivo	Output della fase	Gate di passaggio
Propose	Allineare intento e ambito prima del codice	Cartella in <code>changes/</code> con <code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs	Revisione umana dell'intento
Apply	Implementare senza deviazioni dai vincoli	Codice + test aderenti a <code>design.md</code> e <code>tasks.md</code>	Verifica qualità su MUST/SHALL
Archive	Consolidare la modifica nella verità di sistema	Fusione Delta Specs in <code>specs/</code> + archivio storico della change	Stato aggiornato e audit trail persistente

Logica del ciclo: cattura presto il disallineamento (Propose), esegui in modo deterministico (Apply), consolida nella Source of Truth (Archive).



Google
Developer
Groups

Focalizzazione: il 100% su un compito alla volta

Nel vibe coding il modello divide l'attenzione fra business, architettura e sintassi. Con OpenSpec ogni fase riceve l'intera capacità del modello:

Fase	Focus dell'agente
Propose	100% sull'architettura e le decisioni di design
Apply	100% sulla correttezza e completezza del codice
Archive	100% sulla coerenza documentale e la Source of Truth

Più focus su un compito → Meno errori → Meno cicli di correzione



Google
Developer
Groups

OpenSpec scala col team

- **Orizzontale (branch paralleli):** più agenti su feature diverse, ognuno con la propria cartella `changes/`; il ciclo guida il merge ordinato
- **Verticale (Project Management):** via MCP dialoga con Linear/Jira, il backlog resta allineato allo stato reale del codice
- **Multi-agente:** un Architetto pianifica, un Orchestratore delega, team di agenti specializzati eseguono in parallelo i task atomici

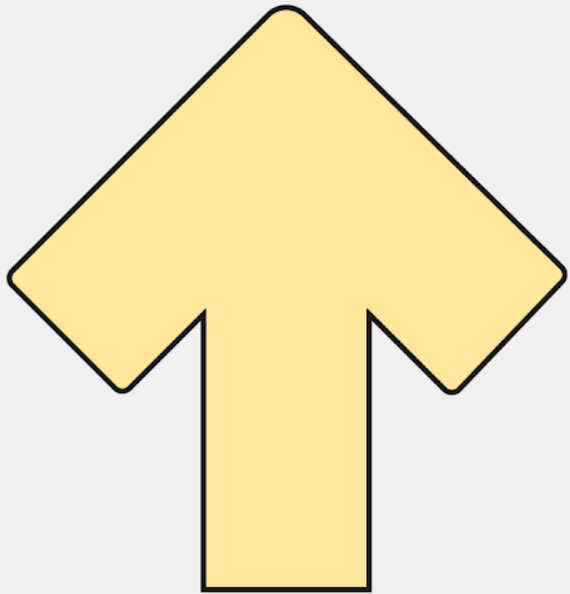


Google
Developer
Groups

05

Dal Team al Legacy

Adozione incrementale sul brownfield
e anti-pattern da evitare



Google
Developer
Groups

Brownfield: adottare senza fermare il mondo

Come si introduce OpenSpec in 100.000 righe di codice senza documentazione? **Non riscrivendo tutto da zero.**

- **Sviluppo just-in-time:** documenta solo ciò che tocchi, partendo dalla prossima feature o dal prossimo bug critico
- **Accumulo organico:** la Source of Truth cresce ad ogni ciclo `Propose → Apply → Archive`
- **Reverse engineering:** `/opsx:onboard` analizza il codice esistente e genera specifiche retroattive da validare col team (opzionale, non un prerequisito)



Google
Developer
Groups

Mitigazione dei rischi: 4 anti-pattern da evitare

Il cimitero delle proposte

Problema: dimenticare `Archive` e lasciare change mai consolidate.

Fix: integrare l'archiviazione nel workflow operativo e nei controlli di qualità.

Il micro-management dell'AI

Problema: scrivere dettagli implementativi nelle specifiche principali.

Fix: tenere il comportamento nelle spec e spostare il "come" in `design.md`.

Burocrazia per modifiche banali

Problema: usare una SDD completa per un cambio minimo e non rischioso.

Fix: applicare progressive rigor e mantenere un approccio leggero.

Ignorare la formattazione Delta

Problema: non usare `ADDED`, `MODIFIED`, `REMOVED` in modo strutturato.

Fix: rispettare i metadati Delta per merge puliti e consolidamento affidabile.



Google
Developer
Groups

06

Perché Tutto Questo Conta

Determinismo, intento persistente e
il ruolo dell'Agentic Engineer



Google
Developer
Groups

Impatto strategico

Determinismo + intento persistente = scalabilità umano-AI

Precisione	Resilienza	Evoluzione del ruolo
Specifiche eseguibili: più accuratezza al primo tentativo, meno rilavorazioni.	La conoscenza architetture non muore nella chat: resta al team e resiste al turnover.	Il valore umano passa dallo "scrivere tutto" al garantire che il codice sia quello necessario.

Sintesi: quando l'implementazione è deterministica e l'intento rimane persistente, il sistema scala senza perdere qualità.



Google
Developer
Groups

Il nuovo professionista: l'Agentic Engineer

Un ingegnere di sistemi agentici che padroneggia la progettazione del contesto, la segmentazione in task atomici e il coordinamento di collaboratori sintetici specializzati.

OpenSpec non riguarda lo scrivere meno codice, ma garantire che il codice generato sia **esattamente quello necessario**.

Il codice è l'output. La specifica è la competenza.



Google
Developer
Groups

Cosa porti a casa da domani

- **Installa:** `npm install -g @fission-ai/openspec`, poi `openspec init` nel tuo progetto esistente
- **Parti piccolo:** documenta solo la prossima feature o il prossimo bug critico, niente Big Bang documentale
- **Scrivi per l'agente:** requisiti EARS (`SHALL` / `SHOULD`) + scenari `GIVEN` / `WHEN` / `THEN` verificabili
- **Separa le fasi:** prima rivedi la proposta (secondi), poi lascia scrivere il codice (l'architettura sbagliata costa giorni)
- **Archivia sempre:** le Delta Specs fuse in `specs/` sono la tua Source of Truth, non la chat



Google
Developer
Groups

Grazie! Domande? Parliamone.



<https://www.baccan.it>

"Smetti di chattare, inizia a governare."



Google
Developer
Groups